

• ENGENHARIA DE SOFTWARE • BIMESTRE 01

Git & GitHub na prática

Aula 05. Hoje a gente abre a pasta `.git` de verdade — e conecta o projeto de vocês ao GitHub.

Revisão rápida

Um arquivo editado duas vezes no VS Code, dois commits, e um `reset --hard` pra reaquecer — o mesmo fluxo da aula passada, de novo com as mãos no teclado.

```
recap-aula-04/

# cria o repositório .git nesta pasta
$ git init
# manda os arquivos pra área de preparação (staging)
$ git add .
# grava um snapshot permanente do que foi preparado
$ git commit -m "Cria anotacoes.txt..."
$ git add .
$ git commit -m "Adiciona segunda linha..."
# --oneline resume cada commit em uma linha só
$ git log --oneline
# desfaz o último commit, mesmo ele já tendo sido salvo
$ git reset --hard HEAD~1
```

Todo esse histórico, até agora, vive só nessa máquina. É esse ponto que a aula de hoje resolve.

cmd → wsl → Desktop

- ! O path da Área de Trabalho muda de PC pra PC — `whoami` no WSL não ajuda, porque o usuário do Linux não é o do Windows. Prefere ver visualmente? Abre o Explorador de Arquivos com `Win` + `E` e navega até a Área de Trabalho — o caminho aparece na barra de endereço lá em cima.

```
cmd.exe

# entra no ambiente Linux (WSL) a partir do cmd
C:\> wsl
# descobre o nome de usuario certo neste PC
$ ls /mnt/c/Users/
# entra na Área de Trabalho desse usuário
$ cd /mnt/c/Users/NOME/Desktop
# cria a pasta do projeto
$ mkdir es-aula-05
# entra na pasta recém-criada
$ cd es-aula-05
```

atenção

Vocês vão receber só isto:

```
es-aula-05/  
  
# -a mostra também os arquivos ocultos (que começam com .)  
$ ls -la  
.git
```

Nenhum `index.html`. Nenhum `css/`. Só a pasta oculta. O projeto inteiro vai "aparecer" a partir dela.

Não vê a pasta `.git` no VS Code? A causa (e o ajuste) tão no fim da aula, no troubleshooting.

O que aconteceu com os arquivos?

```
es-aula-05/  
  
# mostra o que mudou em relação ao último commit  
$ git status  
deleted:    index.html  
deleted:    css/style.css  
deleted:    sobre/sobre.html
```

"Deletados"? Vocês nunca chegaram a ter esses arquivos.

Um comando. Tudo volta.

```
es-aula-05/  
  
# traz de volta os arquivos apagados, usando o que já foi commitado  
$ git restore .  
# ou: git reset --hard main  
$ ls -la  
css index.html sobre  
# --oneline resume cada commit em uma linha só  
$ git log --oneline  
1878ee0 adiciona roteiro-aula-05.md ao repositório  
f56180f preenche o conteúdo da página sobre...  
ae17b87 adiciona texto de boas-vindas na home...  
99d7856 cria estrutura inicial da página...
```

Ninguém digitou nada disso agora – veio tudo de dentro da pasta `.git` que já estava ali.

Tem arquivo que **não volta**

Não é o `.gitignore` em si que fica de fora — é **o que ele marca como ignorado**. Esses arquivos nunca chegam a ser commitados.

Por isso não voltam: nunca existiram como snapshot dentro do `.git` — só ficaram soltos no disco.

A lição do dia, em uma frase: **só o que foi commitado mora dentro do `.git`**.

HEAD

— pra onde eu olho agora

es-aula-05/

mostra pra qual branch o HEAD está apontando agora

\$ cat .git/HEAD

ref: refs/heads/main

HEAD não guarda um commit direto — é um ponteiro pra **outro ponteiro**: a branch atual.

Ponteiro, resumindo: um valor que não guarda o dado — só aponta pra onde o dado de verdade está.

Uma branch é só um arquivo

```
es-aula-05/  
  
# mostra o hash do commit que a branch main aponta  
$ cat .git/refs/heads/main  
1878ee023677560b204d2427f3a3c900c1c0986b  
# mesma pergunta, respondida pelo próprio Git  
$ git rev-parse main  
1878ee023677560b204d2427f3a3c900c1c0986b
```

Mesmo hash do topo do `git log`. Uma branch é um texto com um hash dentro — nada mais místico que isso.

config – antes do GitHub

```
es-aula-05/  
  
# mostra as configurações locais deste repositório  
$ cat .git/config  
[core]  
    repositoryformatversion = 0  
    filemode = true  
    bare = false
```

Guarda esse arquivo na cabeça — ele ganha uma seção nova daqui a pouco.

GitHub não é "o git"

É um serviço de hospedagem de repositórios Git na internet. Poderia ser GitLab, Bitbucket, ou um servidor da faculdade.

Na linguagem do Git, esse endereço externo se chama `remote` – `origin` é só o apelido mais comum.

Criando o repositório no GitHub

- Em `github.com`, clica no `+` (canto superior direito) → `New repository`
- Dá um nome e escolhe `Public` ou `Private`
- Não marca `Add a README` — o repo tem que nascer vazio
- Confirma em `Create repository`

Na página do repo recém-criado: botão verde `Code` → aba `SSH` → copia a URL. É o que entra no próximo comando.

Chave SSH: cadeado e chave

A pública é o cadeado que vocês entregam pro GitHub guardar. A privada é a única chave que abre — e fica só com vocês, nunca sai da máquina.

É por isso que o GitHub aceita um `git push` vindo do computador de vocês sem pedir usuário e senha toda vez.

Gerando e cadastrando

```
# gera o par de chaves SSH (pública e privada)
$ ssh-keygen -t ed25519 -C "voce@email.com"
# mostra a chave pública, pra copiar
$ cat ~/.ssh/id_ed25519.pub
# testa se o GitHub reconhece a chave
$ ssh -T git@github.com
Hi usuario! You've successfully authenticated...
```

- Clica na sua **foto de perfil** (canto superior direito) → `Settings`
- No menu lateral, entra em `SSH and GPG keys`
- Botão verde `New SSH key`
- Em `Title`, dá um nome pra identificar essa máquina (ex: "meu notebook")
- Em `Key`, cola o conteúdo copiado do `id_ed25519.pub`
- Confirma em `Add SSH key`

Vendo essa mensagem, a autenticação já está pronta — o próximo `git push` não vai pedir senha nenhuma.

Cadastrando o remote

```
es-aula-05/  
  
# cadastra o endereço remoto com o apelido "origin"  
$ git remote add origin git@github.com:usuario/repo.git  
# confirma que o endereço foi salvo no config  
$ cat .git/config  
[core] ...  
[remote "origin"]  
    url = git@github.com:usuario/repo.git
```

`git remote add` não fez mágica de rede — só escreveu duas linhas num arquivo texto.

push e pull

```
# enviar  
git push -u origin main
```

```
# receber  
git pull
```

Decompondo `git push -u origin main`: `origin` é o **remote** (pra onde vai), `main` é o **branch** (o que vai), e `-u` marca os dois como o padrão — só precisa na primeira vez. Depois disso, `git push` e `git pull` sozinhos já sabem pra onde ir.

Pensa em calistenia: `push` é flexão — você empurra o que é seu pra fora. `pull` é barra — você puxa o que tá lá fora pra perto de você.

troubleshooting

Cadê o `.git` no VS Code?

- `Ctrl` + `,` abre as Settings
- Buscar `files.exclude`
- Desmarcar a entrada `**/.git`

```
{  
  "files.exclude": {  
    "**/.git": false  
  }  
}
```

S0, terminal, WSL, Git, GitHub.

A base da primeira parte do ano está fechada. A seguir: organizar o que se constrói — Python e orientação a objetos.